MOC: dbt (MOC)
MOC: Cursos

Introdução ao dbt Fundamentals

- O curso apresenta o **dbt** como uma ferramenta voltada para **transformação de dados** e **engenharia de analytics**.
- O objetivo é desenvolver familiaridade prática com o dbt e com a estrutura de um projeto.

Objetivos do curso

Ao final do curso, você deverá ser capaz de:

- Entender os **conceitos centrais do dbt**.
- Compreender suas **aplicações práticas**.
- Entender por que o dbt é relevante para profissionais de dados.
- Navegar por um **projeto dbt**.
- Escrever **modelos básicos de dbt**.
- Executar modelos dbt de forma eficaz.
- Ganhar familiaridade com uma ferramenta amplamente usada em engenharia de dados.

Conhecimentos prévios

É recomendado ter:

- Experiência prévia com **SQL**.
- Familiaridade básica com **linha de comando**.
  Também ajuda conhecer:
- **Pipelines de dados**.
- **Modelagem de dados**.
- Esses conhecimentos não são o foco principal do curso, mas facilitam o acompanhamento dos exercícios.

Ambiente de aprendizagem interativo

- Os exercícios utilizam uma **IDE (Ambiente de Desenvolvimento Integrado)**.
- Uma IDE reúne diferentes ferramentas em uma única interface para facilitar o desenvolvimento.
- O **Visual Studio Code (VS Code)** é um exemplo de IDE.
- A IDE usada no curso possui três componentes principais:
Componente	Função

Seletor de arquivos	Navegar e mover arquivos dentro da estrutura de diretórios
Editor de código	Abrir, ler e modificar arquivos
Terminal	Executar comandos necessários nos exercícios

Seletor de arquivos

- Fica no lado esquerdo da IDE.
- Permite navegar pela estrutura de diretórios.
- Pode ser usado para selecionar e mover arquivos.

Editor de código

- Fica na parte superior direita da IDE.
- Permite alternar entre arquivos abertos.
- É usado para ler e modificar o conteúdo dos arquivos.
- As alterações são **salvas automaticamente**.

Terminal

- Fica na parte inferior direita da IDE.
- É usado para executar comandos necessários durante os exercícios.
- Seu uso ficará mais claro à medida que o curso avançar.

> **Resumo rápido**
> - O curso ensina os fundamentos do **dbt**, incluindo navegação em projetos, criação de modelos e execução.
> - É recomendado conhecer **SQL**, linha de comando e conceitos básicos de pipelines ou modelagem de dados.
> - Os exercícios usam uma IDE com **seletor de arquivos, editor de código e terminal**.

Criando um projeto dbt

O que é um projeto dbt

- Um projeto dbt reúne os componentes necessários para trabalhar com dados dentro do dbt.
- É uma **coleção estruturada de arquivos** que define como os dados serão transformados e organizados.
- A configuração do projeto pode incluir:
  - Nome do projeto.
  - Nomes das pastas.
  - Fontes de dados.

    - Destinos de dados.
    - Data warehouses utilizados.
    - Consultas **SQL**.
    - Templates usados para acessar e transformar os dados.
    - Documentação sobre os dados e seus relacionamentos.
- Um projeto dbt é implementado como uma **estrutura de pastas**.
- Essa estrutura pode ser facilmente:
    - Copiada.
    - Modificada.
    - Colocada em controle de versão.

Como criar um novo projeto

- Para criar um projeto dbt, usamos:

```
dbt init
```

- Ao executar `dbt init`, o dbt pergunta:
    - O nome do projeto.
    - O tipo de banco de dados ou data warehouse que será usado.
- Também é possível informar o nome diretamente:

```
dbt init nomedoprojeto
```

- Nesse caso, o dbt pergunta apenas o tipo de banco de dados.
- O `dbt init` cria:
    - A pasta principal do projeto.
    - Subpastas.
    - Arquivos de configuração.
- Exemplo citado no curso:

```
dbt init test_project
```

- Nesse exemplo, o projeto usa **DuckDB** como banco de dados.

Profiles do projeto

- Um **profile** representa um cenário de implantação.
- Exemplos de cenários:
    - Desenvolvimento.
    - Homologação ou teste.

    - Produção.
- Um projeto dbt pode ter vários profiles.
- Isso permite usar diferentes configurações de warehouse conforme o cenário de implantação.
- Os profiles são definidos no arquivo:

```
profiles.yml
```

- Esse arquivo precisa ser criado para novos projetos.
- Um `profiles.yml` pode conter, por exemplo:
    - `dev`
    - `prod`
- A opção `target` define qual configuração será usada como padrão.
- Exemplo citado:

```
target: dev
```

- No exemplo do curso:
    - **DuckDB** é usado para desenvolvimento e testes locais.
    - **Snowflake** é usado como exemplo de ambiente de produção, onde outras pessoas podem precisar acessar os dados.

YAML

- YAML significa **Yet Another Markup Language**.
- É um formato de arquivo baseado em texto.
- A **indentação por espaços é importante**, de forma parecida com Python.
- YAML é muito usado para arquivos de configuração.
- Uma das vantagens é ser relativamente legível para humanos.
- É necessário ter cuidado ao editar YAML, porque erros de indentação podem alterar ou invalidar a configuração.
- Exemplo citado no curso:

```
dev:
prod:
```

- `dev:` e `prod:` devem estar no mesmo nível de indentação quando representam elementos equivalentes.
- O curso fornece esqueletos de YAML, mas é importante entender as regras de formatação ao criar arquivos manualmente.

DuckDB

- **DuckDB** é um banco de dados:
  - Open source.
  - Sem servidor.
- Seu funcionamento é comparado ao **SQLite**.
- Diferente de PostgreSQL ou MySQL, não exige um processo de servidor separado.
- É projetado para análise de dados e cenários semelhantes a data warehouses.
- Possui execução vetorizada, contribuindo para seu desempenho.
- O curso usa DuckDB porque:
  - É fácil de usar.
  - Funciona com dbt.
  - É adequado para desenvolvimento e testes locais.
- O DuckDB pode ser usado no computador local ou em um Workbook da DataCamp.

> **Resumo rápido**
>
> - Um projeto dbt é uma estrutura organizada de arquivos com configurações, SQL, templates, fontes, destinos e documentação.
> - `dbt init` cria a estrutura inicial de um projeto dbt.
> - `profiles.yml` define configurações de ambientes como `dev` e `prod`, usando `target` para escolher o padrão.
> - YAML exige atenção à indentação, e o curso usa DuckDB por ser simples, local e adequado para análise.

Workflow do dbt

Fluxo de trabalho com dbt

- O workflow do dbt pode variar conforme a necessidade, mas normalmente segue estas etapas:
  1. Criar um projeto com `dbt init` ou copiar um projeto existente.
  2. Definir ou atualizar as configurações no `profiles.yml`.
  3. Criar e trabalhar com **modelos de dados**.
  4. Executar os modelos com `dbt run`.
  5. Verificar e testar os dados.
  6. Corrigir problemas, se necessário.
  7. Repetir o processo quando houver novas fontes ou alterações.
- A maior parte do desenvolvimento com dbt acontece na definição e uso de **modelos**.
- Um modelo pode ser entendido, inicialmente, como uma versão transformada dos dados brutos armazenados no data warehouse.

- O comando `dbt run` pega o SQL escrito no projeto, adapta conforme o destino de implantação e executa as transformações.

dbt run

- `dbt run` executa as transformações de dados e envia as atualizações para o warehouse.

```
dbt run
```

- Deve ser executado quando:
  - Há mudanças nos modelos.
  - O processo precisa ser materializado.
- A saída do `dbt run` deve ser analisada porque mostra:
  - Informações de versão.
  - Progresso da execução.
  - Sucesso ou falha de cada etapa.
  - Processamento dos modelos.

Materialização

- No dbt, **materializar** significa executar as transformações sobre os dados de origem e armazenar o resultado como:
  - Tabela.
  - View.
- Esse conceito é importante porque o dbt pode criar diferentes tipos de objetos no banco conforme a configuração.

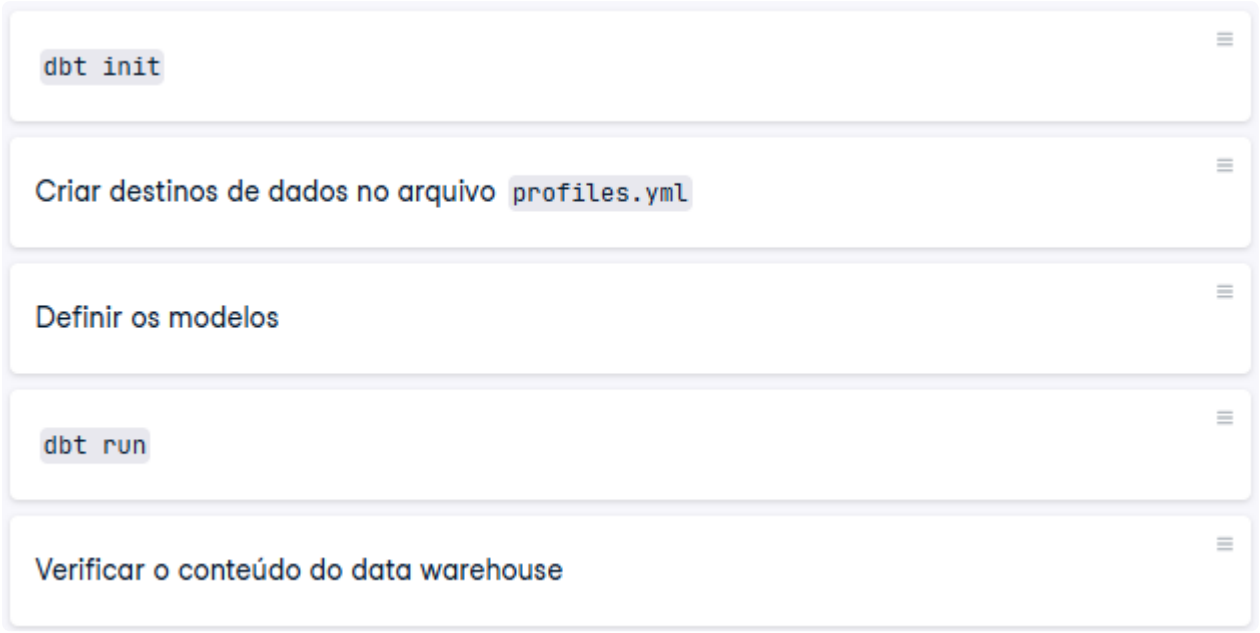
Tabela vs View

Aspecto	Tabela	View
Armazena dados	Sim	Não
Ocupa espaço	Conforme o volume dos dados	Muito pouco
Atualização	Apenas quando os dados são alterados	Resultado gerado quando consultada
Comportamento	Objeto físico com dados armazenados	Consulta que se comporta como uma tabela
Definição comum	Dados inseridos ou transformados	`SELECT` sobre uma ou mais tabelas

- Uma **tabela** armazena fisicamente os dados dentro do banco ou data warehouse.

- O conteúdo da tabela só muda quando algum comando altera esses dados.
- Uma **view** pode ser consultada como uma tabela, mas não armazena os dados da mesma forma.
- O resultado de uma view é gerado quando ela é consultada.
- O dbt pode criar tanto tabelas quanto views.
- A implementação específica de cada uma depende do banco de dados utilizado.

> **Resumo rápido**
> - O workflow do dbt envolve criar o projeto, configurar `profiles.yml`, desenvolver modelos, executar com `dbt run` e testar os resultados.
> - `dbt run` executa as transformações e materializa os modelos no warehouse.
> - Materialização significa transformar os dados e gerar objetos como tabelas ou views.
> - Tabelas armazenam dados fisicamente; views representam consultas executadas sobre outros dados.



Modelos de dados no dbt

O que é um modelo de dados?

- Não existe uma definição única e rígida para **modelo de dados**.
- O significado depende do contexto.
- Em essência, um modelo de dados define:
  - A organização lógica dos dados.
  - A interpretação de um conjunto de dados.
  - Como os componentes desse conjunto se relacionam.
- Um modelo pode representar:

    - Uma tabela de banco de dados.
    - Um DataFrame.
    - Pedidos.
    - Clientes.
    - Terremotos de uma região.
    - Características de animais.
- Um dos objetivos principais é permitir que diferentes pessoas compreendam e discutam os dados de forma compartilhada.

Exemplo de modelo de dados

- Um modelo simples pode conter:
    - Espécie do animal.
    - Número de pernas.
    - Se é venenoso.
- Esses atributos formam uma maneira específica de representar o animal.
- Poderiam ser escolhidos outros atributos.
- A escolha dos atributos influencia a efetividade do modelo.
- Sempre existem **trade-offs** ao definir um modelo, como:
    - Complexidade.
    - Espaço necessário.
    - Quantidade de atributos.

O que é um modelo no dbt?

- No dbt, um modelo representa algo mais específico.
- Ele representa as **transformações realizadas sobre os dados brutos de origem**.
- Essas transformações são normalmente escritas em **SQL**.
- Versões mais novas do dbt também podem usar Python, mas isso não é abordado neste curso.
- Cada modelo contém uma consulta `SELECT`.
- O objetivo dessa consulta é transformar os dados de origem no formato desejado.
- Cada modelo é salvo em um arquivo com extensão:

```
.sql
```

- O dbt utiliza esses arquivos automaticamente em comandos como:

```
dbt run
```

Criando um modelo dbt simples

- Primeiro, crie um diretório dentro da pasta `models` do projeto.
- Esse diretório pode ter qualquer nome, mas é recomendável manter uma convenção consistente.
- Depois, crie um arquivo `.sql`.
- O arquivo pode ser criado:
  - Pela linha de comando.
  - Com uma ferramenta como `touch`.
  - Em um editor de texto.
- Em seguida, adicione a consulta SQL ao arquivo.
- Exemplo:

```
SELECT
    first_name,
    last_name
FROM source_table;
```

- Por fim, execute:

```
dbt run
```

- Isso materializa o modelo.

Parquet

- **Parquet** é um formato binário colunar.
- É usado para armazenar dados de forma eficiente.
- É amplamente utilizado para compartilhar e distribuir conjuntos de dados.
- O formato colunar se diferencia de formatos baseados em linhas.
- O DuckDB consegue ler arquivos Parquet diretamente, sem necessidade de importação prévia.

Lendo Parquet com DuckDB

- O DuckDB pode usar a função `read_parquet`.
- Exemplo:

```
SELECT *
FROM read_parquet('filename.parquet');
```

- Também é possível referenciar diretamente um arquivo Parquet usando aspas simples.

Atualizando modelos dbt

Por que atualizar modelos?

- Uma das vantagens do dbt é permitir alterações em um projeto sem precisar recriar consultas ou modelos do zero.
- Alguns motivos comuns para atualizar modelos:
  - **Tarefas iterativas**: quando os requisitos do projeto mudam ou ainda não foram totalmente implementados.
  - **Correção de bugs**: problemas em consultas SQL podem exigir ajustes nos modelos.
  - **Migração de dados**: mudanças nas fontes ou nos destinos dos dados podem exigir atualizações no projeto.

Fluxo de atualização

- Um fluxo comum de atualização de um projeto dbt pode seguir estas etapas:
  1. Baixar ou atualizar o projeto a partir de um sistema de controle de versão, como Git.
  2. Localizar o arquivo de modelo que precisa ser alterado.
  3. Atualizar a consulta SQL.
  4. Executar o projeto novamente com `dbt run`.
  5. Verificar se os resultados estão corretos.
  6. Registrar as alterações no controle de versão.
- Exemplo usando Git:

```
git clone dbt_project
cd dbt_project
```

- O Git não é obrigatório para usar dbt, mas ajuda a acompanhar:
  - Mudanças.
  - Atualizações.
  - Modificações no projeto.
- A atualização de um modelo pode envolver:
  - Alterar diretamente a consulta SQL.
  - Criar uma subconsulta.
  - Modificar outras partes do arquivo `.sql`.

Aplicando as alterações

- Depois de modificar os modelos, execute:

```
dbt run
```

- Esse comando aplica as mudanças ao projeto.

- Em alguns casos, uma alteração maior pode exigir uma atualização completa do modelo:

```
dbt run -f
```

- A atualização completa pode ser útil quando:
  - Ocorre algum erro após a atualização.
  - Os resultados não aparecem como esperado.
- Esse processo pode levar mais tempo do que uma atualização simples, dependendo dos dados e dos modelos.

Arquivos YAML

- Além dos arquivos `.sql`, também é possível atualizar arquivos YAML do projeto.
- Dois arquivos comuns são:
  - `dbt_project.yml`
  - `model_properties.yml`

dbt_project.yml

- O arquivo `dbt_project.yml` contém configurações aplicadas ao projeto inteiro.
- Pode incluir:
  - Nome do projeto.
  - Versão.
  - Localização dos diretórios.
  - Configurações de materialização.
- As configurações de materialização podem definir se os modelos serão criados como:
  - Tabelas.
  - Views.
  - Outros tipos de objetos.
- As configurações desse arquivo são aplicadas globalmente ao projeto.
- Existe apenas um arquivo:

```
dbt_project.yml
```

por projeto.

model_properties.yml

- O arquivo `model_properties.yml` é usado para configurações e informações específicas dos modelos.
- Pode incluir:

    - Descrições.
    - Informações de documentação.
    - Outras propriedades relacionadas aos modelos.
- Apesar do nome `model_properties.yml`, o arquivo pode ter qualquer nome.
- Os requisitos são:
    - Estar dentro do diretório `models/` ou em algum de seus subdiretórios.
    - Ter a extensão `.yml`.
- Um projeto pode conter vários arquivos `.yml` desse tipo.

Escopo	Arquivo
Projeto	`profiles.yml`
Projeto	`dbt_project.yml`
Somente o modelo	`test_model.sql`
Somente o modelo	`model_properties.yml`
Somente o modelo	`models/test_model.yml`

Documentação no dbt

Por que documentar?

- Projetos de dados costumam conter anotações no código e nas consultas para ajudar engenheiros e desenvolvedores.
- Porém, os dados também são consumidos por outras pessoas que não precisam ou não devem acessar o código-fonte.
- A documentação ajuda a disponibilizar informações importantes sem exigir acesso direto ao projeto.
- Centralizar a documentação facilita a consulta e evita depender de canais dispersos, como e-mails.
- A documentação também pode registrar:
    - Atualizações e mudanças nos feeds de dados.
    - Exemplos de uso.
    - Sugestões de utilização dos dados.
    - Detalhes de SLA (**Service Level Agreement / Acordo de Nível de Serviço**).
- Um SLA pode indicar:
    - Com que frequência os dados são atualizados.
    - Quais garantias existem sobre a disponibilidade dos dados.

Criando documentação no dbt

- O dbt permite adicionar documentação ao projeto de forma automática.
- É possível documentar:

    - Modelos.
    - Descrições gerais dos modelos.
    - Colunas individuais.
    - Testes.
    - Validações.
    - Informações do data warehouse gerado.
- A documentação também pode mostrar a **linhagem dos dados**.
- Essa linhagem pode ser representada como um **DAG (Directed Acyclic Graph / Grafo Acíclico Dirigido)**.
- O DAG mostra o fluxo dos dados:
    - Das tabelas-fonte.
    - Passando pelas transformações.
    - Até tabelas agregadas ou outros resultados finais.
- A documentação do dbt também pode exibir:
    - Tipos de dados das colunas.
    - Tamanhos dos dados gerados.

Gerando a documentação

- O dbt possui o subcomando:

```
dbt docs
```

- Para consultar a ajuda:

```
dbt docs -h
```

- O principal comando para gerar a documentação é:

```
dbt docs generate
```

- Esse comando:
    - Percorre o conteúdo do projeto.
    - Gera a documentação automaticamente.
    - Formata o resultado como um site estático.
- Como a documentação muda conforme novos modelos, testes e outros componentes são adicionados, o ideal é executá-la depois de:

```
dbt run
```

- Fluxo comum:

```
dbt run
dbt docs generate
```

Acessando a documentação

- Depois de gerar a documentação, é necessário abri-la em um navegador.
- Uma opção local é:

```
dbt docs serve
```

- Esse comando inicia um servidor web local para visualizar a documentação.
- É adequado para desenvolvimento local.
- Não é recomendado como solução de hospedagem pública ou de produção.
- Outras opções de hospedagem mencionadas:
  - dbt Cloud.
  - Amazon S3.
  - Nginx.
  - Apache.
  - Outros servidores web.

O que a documentação pode mostrar

- Modelos.
- Descrições.
- Informações sobre colunas.
- Testes e validações.
- Tipos de dados.
- Tamanho dos dados.
- Linhagem dos dados.
- DAG do projeto.

Ordem	Etapa
1	Adicionar documentação a modelos, testes etc.
2	Usar `dbt run` para gerar os modelos
3	Usar `dbt docs generate` para gerar os arquivos de documentação
4	Copiar o conteúdo para o serviço de hospedagem
5	Acessar a documentação pelo navegador da Web

Jinja no dbt

O que é um template?

- Um **template** é um formato pré-definido que permite ao dbt substituir informações automaticamente.
- Normalmente, templates são arquivos de texto.
- Eles podem ser editados em editores comuns.
- O objetivo principal é facilitar:
  - Reuso.
  - Acesso.
  - Padronização.

O que é Jinja?

- **Jinja** é uma linguagem de template baseada em texto.
- É usada em várias ferramentas além do dbt, como:
  - Django.
  - Flask.
- No dbt, o conteúdo Jinja é colocado entre duas chaves:

```
{{ ... }}
```

- Quando o dbt é executado, ele substitui o conteúdo entre as chaves pelo resultado correspondente.
- O dbt disponibiliza várias funções Jinja para uso nos projetos.
- Jinja permite tornar o projeto mais dinâmico.
- Pode ajudar a trabalhar com:
  - Diferentes locais de origem.
  - Diferentes destinos de dados.
  - Repetições de código.
  - Loops `for`.

Funções Jinja no dbt

- Algumas funções citadas:
Função	Uso
`ref`	Acessar modelos pelo nome
`config`	Acessar ou definir configurações
`docs`	Acessar conteúdos de documentação
- Jinja também pode oferecer recursos como:
  - Cálculos matemáticos.
  - Estruturas de loop.

Exemplo sem Jinja

- Uma consulta SQL pode repetir a mesma lógica para várias colunas:

```
SELECT
    COALESCE(start_date, '1900-01-01') AS start_date,
    COALESCE(update_date, '1900-01-01') AS update_date,
    COALESCE(end_date, '1900-01-01') AS end_date
FROM source_table;
```

- Nesse caso, a estrutura é praticamente a mesma para todas as colunas.

Exemplo com Jinja

- Com Jinja, é possível usar um loop `for` para reduzir a repetição:

```
SELECT
{% for column in ['start_date', 'update_date', 'end_date'] %}
    COALESCE({{ column }}, '1900-01-01') AS {{ column }}
{% endfor %}
FROM source_table;
```

- O loop percorre a lista de colunas.
- A variável `column` é substituída pelo nome correspondente.
- O resultado final representa a mesma lógica da consulta SQL repetitiva.

Por que usar Jinja?

- Reduz repetição de código.
- Facilita o reuso.
- Torna modelos mais dinâmicos.
- Ajuda a manter consultas mais simples quando existe lógica repetitiva.

Modelos hierárquicos no dbt

O que é uma hierarquia no dbt?

- Uma **hierarquia** representa as dependências entre os componentes de um projeto dbt.
- Ela mostra a relação entre:
  - Dados de origem.
  - Dados transformados.
- Essa estrutura também é chamada de:
  - **DAG (Directed Acyclic Graph / Grafo Acíclico Direcionado)**.
  - Grafo de linhagem.

- Embora DAG também exista em ferramentas como Spark e Airflow, aqui o conceito está sendo usado conforme a implementação do dbt.
- O objetivo principal da hierarquia é permitir que o dbt determine a ordem correta de construção e atualização dos modelos.

Por que a hierarquia é importante?

- Modelos podem depender de outros modelos.
- Exemplo:

```
taxi_rides_raw
├── avg_fare_per_day
└── total_creditcard_riders_per_day
```

- `avg_fare_per_day` e `total_creditcard_riders_per_day` dependem de `taxi_rides_raw`.
- O dbt precisa construir primeiro:

```
taxi_rides_raw
```

- Depois, pode construir os modelos dependentes.
- Sem essa hierarquia, os modelos poderiam ser executados em uma ordem incorreta.
- No exemplo do curso, uma execução por ordem alfabética tentaria construir `avg_fare_per_day` antes de `taxi_rides_raw`, causando falha.

Como as hierarquias são definidas?

- As dependências são definidas dentro dos arquivos `.sql` dos modelos.
- O dbt usa Jinja para indicar essas relações.
- A função principal usada é:

```
{{ ref('nome_do_modelo') }}
```

- Em vez de referenciar diretamente uma tabela:

```
SELECT *
FROM taxi_rides_raw;
```

- Usa-se:

```
SELECT *
FROM {{ ref('taxi_rides_raw') }};
```

- Isso informa ao dbt que o modelo atual depende de `taxi_rides_raw`.

Função `ref`

- `ref()` é usada para referenciar outro modelo pelo nome.
- Exemplo:

```
{{ ref('taxi_rides_raw') }}
```

- O dbt usa essa referência para:
  - Identificar dependências.
  - Construir o DAG.
  - Determinar a ordem de execução.
  - Substituir a referência pelo nome real da tabela no SQL gerado.

Executando modelos hierárquicos

- Depois de definir as dependências com `ref()`, execute:

```
dbt run
```

- O dbt analisa as dependências antes de materializar os modelos.
- Os modelos são executados na ordem necessária para que cada dependência esteja disponível antes dos modelos a jusante.

Testes no dbt

O que é um teste?

- No dbt, um **teste** é uma asserção ou validação aplicada a objetos do projeto.
- Os testes podem ser usados em:
  - Modelos.
  - Sources.
  - Seeds.
- O objetivo é verificar se os dados estão conforme o esperado.
- Exemplos de validações:
  - Valores nulos.
  - Valores dentro de um intervalo ou lista esperada.
  - Relacionamentos entre dados.
  - Regras personalizadas de negócio ou lógica.

Tipos de testes

- O dbt possui três tipos de testes:
  - **Built-in**: testes nativos e predefinidos.
  - **Singular**.
  - **Generic**.
- Os testes singular e generic serão abordados posteriormente.

Testes nativos

- O dbt possui quatro testes nativos principais:

	Teste
	`unique`
	`not_null`
	`accepted_values`
	`relationships`

- `accepted_values` utiliza a opção:

```
values:
```

- `relationships` utiliza opções como:

```
to:
field:
```

Onde aplicar os testes

- Testes de modelos são definidos em arquivos YAML dentro do diretório `models/`.
- Um exemplo de nome é:

```
model_properties.yml
```

- O arquivo pode ter outro nome, desde que seja um arquivo `.yml` no local apropriado.
- Os testes são definidos dentro das propriedades das colunas.
- Exemplo:

```
models:
  - name: taxi_rides_raw
    columns:
      - name: tpep_pickup_datetime
        tests:
          - not_null
      - name: payment_type
```

```
        tests:
          - not_null
          - accepted_values:
              values: [1, 2, 3, 4, 5, 6]
```

- Nesse exemplo:
  - `tpep_pickup_datetime` não pode conter valores nulos.
  - `payment_type` não pode ser nulo.
  - `payment_type` deve ter valores entre `1` e `6`.

Executando testes

- Para testar o projeto inteiro:

```
dbt test
```

- Para testar um modelo específico:

```
dbt test --select modelname
```

- Exemplo:

```
dbt test --select customers
```

- A saída informa quais testes:
  - Passaram.
  - Falharam.

Investigando falhas

- Para entender por que um teste falhou, pode ser necessário consultar o SQL compilado.
- Os arquivos compilados normalmente ficam em:

```
target/compiled/projectname/models/
```

- No exemplo do curso:

```
target/compiled/nyc_yellow_taxi/models/model_properties.yml/
```

- Dentro desse local, procure o arquivo `.sql` correspondente ao teste que falhou.
- Depois:
  1. Copie o SQL compilado.

2. Execute no cliente do banco de dados.
3. Verifique quais linhas causaram o problema.

- Existem duas formas de corrigir:
  - Remover ou alterar os dados manualmente.
  - Atualizar os scripts dos modelos para tratar o problema.
- Atualizar os scripts tende a ser uma solução mais sustentável.
- Depois da correção, execute novamente:

```
dbt run
dbt test
```

Testes singulares no dbt

O que é um teste singular?

- Um **teste singular** é a forma mais simples de teste de dados personalizado no dbt.
- Ele é escrito como uma consulta **SQL**.
- A consulta deve retornar as **linhas que falharam** na condição esperada.
- Testes singulares são arquivos `.sql` armazenados no diretório:

```
tests/
```

Exemplo de teste singular

- Considere uma tabela de pedidos com colunas como:
  - `subtotal`
  - `order_total`
- Queremos validar que:

```
order_total >= subtotal
```

- Como o teste deve retornar as linhas que falham, a consulta precisa buscar o contrário:

```
SELECT *
FROM orders
WHERE order_total < subtotal;
```

- Se essa consulta retornar linhas, significa que existem registros inválidos.
- Um nome descritivo pode ser:

```
assert_order_total_gte_subtotal.sql
```

- O nome pode ser qualquer um, mas nomes claros facilitam:
  - Logs.
  - Mensagens de erro.
  - Debug.

Teste singular com Jinja

- Testes singulares também podem usar Jinja.
- A função mais comum é `ref()`.
- Em vez de escrever diretamente o nome da tabela:

```
SELECT *
FROM orders
WHERE order_total < subtotal;
```

- Pode-se usar:

```
SELECT *
FROM {{ ref('orders') }}
WHERE order_total < subtotal;
```

- `ref()` substitui o nome correto do objeto quando o projeto é compilado para o data warehouse de destino.
- Outras funções Jinja também podem ser usadas, como `source()`.
- As substituições são feitas quando o teste é executado.
- Se o profile do dbt mudar, o projeto deve ser executado novamente antes dos testes.

Debug de testes

- O fluxo de debug é semelhante ao desenvolvimento de modelos.
- Uma abordagem recomendada:
  1. Criar e testar inicialmente a consulta em um editor SQL.
  2. Corrigir possíveis problemas de SQL.
  3. Salvar a consulta no arquivo `.sql` apropriado.
  4. Garantir que o teste tenha um nome único.
  5. Executar o teste.
  6. Corrigir erros conforme necessário.
- Embora o dbt permita nomes duplicados, isso dificulta identificar qual teste passou ou falhou.

Executando um teste específico

- Para executar apenas um teste:

```
dbt test --select testname
```

- Exemplo:

```
dbt test --select assert_order_total_gte_subtotal
```

- Usar `--select` é especialmente útil em projetos grandes.
- Isso evita executar todos os testes e pode reduzir significativamente o tempo de execução.

Testes reutilizáveis no dbt

O que é um teste reutilizável?

- Um **teste reutilizável**, também chamado de **teste genérico**, pode ser aplicado em várias situações.
- Ele funciona de forma parecida com um teste nativo do dbt, mas permite validar qualquer condição que possa ser expressa em SQL.
- É criado usando **Jinja**.
- O arquivo deve ser salvo como `.sql` dentro de:

```
tests/generic/
```

- Mesmo sendo reutilizável, o teste precisa ser declarado para cada modelo ou coluna em que será usado no arquivo:

```
model_properties.yml
```

Criando um teste genérico

- Um teste genérico normalmente recebe pelo menos dois argumentos:
  - `model`
  - `column_name`
- A primeira linha usa a sintaxe:

```
{% test testname(model, column_name) %}
```

- O nome `testname` deve ser substituído pelo nome real do teste.
- Exemplo: criar um teste que verifique se os valores de uma coluna são maiores que `0`.

```
{% test check_gt_0(model, column_name) %}
SELECT *
FROM {{ model }}
WHERE {{ column_name }} <= 0
{% endtest %}
```

- Nesse teste:
  - `model` representa o modelo ou tabela.
  - `column_name` representa a coluna validada.
  - A consulta retorna as linhas que **falham** na condição.
- O teste termina com:

```
{% endtest %}
```

Aplicando o teste ao modelo

- O teste é aplicado no `model_properties.yml`.
- Exemplo:

```
models:
  - name: taxi_rides_raw
    columns:
      - name: tpep_pickup_datetime
        tests:
          - not_null
      - name: total_fare
        tests:
          - check_gt_0
```

- Nesse caso:
  - `taxi_rides_raw` é passado como `model`.
  - `total_fare` é passado como `column_name`.
- O teste será executado com:

```
dbt test
```

Parâmetros extras

- Testes genéricos podem receber parâmetros adicionais.
- A sintaxe pode ficar assim:

```
{% test check_columns_unequal(model, column_name, column_name2) %}
SELECT *
```

```
FROM {{ model }}
WHERE {{ column_name }} = {{ column_name2 }}
{% endtest %}
```

- Nesse exemplo:
  - `model` representa o modelo.
  - `column_name` representa a primeira coluna.
  - `column_name2` representa a segunda coluna.
- É possível adicionar vários parâmetros, mas é recomendado evitar muitos argumentos obrigatórios.

Aplicando parâmetros extras

- Parâmetros adicionais são informados no YAML abaixo do nome do teste.

```
models:
  - name: taxi_rides_raw
    columns:
      - name: column_1
        tests:
          - check_columns_unequal:
              column_name2: column_2
```

- A indentação é importante em YAML.

Teste singular vs teste genérico

Teste singular	Teste genérico
Criado para uma validação específica	Criado para ser reutilizado
Arquivo `.sql` em `tests/`	Arquivo `.sql` em `tests/generic/`
Consulta SQL direta	Usa template Jinja
Normalmente ligado a um caso específico	Pode ser aplicado a vários modelos e colunas
Executado diretamente como teste	Precisa ser declarado no YAML para cada uso

Sources no dbt

O que é uma source?

- No dbt, **sources** representam dados que já foram carregados pelo processo de **EL**.
- Elas adicionam informações extras sobre dados que já existem no data warehouse.

- O dbt atua apenas na etapa de **Transformação** do processo ELT.
- Sources ajudam principalmente em:
  - Linhagem de dados.
  - Testes.
  - Documentação.

Linhagem de dados

- A principal função das sources é fornecer informações de **linhagem**.
- Linhagem de dados descreve o fluxo dos dados dentro do data warehouse.
- Isso ajuda em:
  - Validação.
  - Solução de problemas.
  - Confiança nos dados.
  - Entendimento da origem dos dados.
- Para acessar uma source, usamos a função Jinja:

```
{{ source('source_name', 'table_name') }}
```

- O uso é semelhante ao de `ref()`.
- Exemplo:

```
{{ source('raw', 'orders') }}
```

- Nesse caso:
  - `raw` representa o nome da source/schema.
  - `orders` representa a tabela.
- O uso de `source()` simplifica o acesso às tabelas sem depender diretamente da sintaxe específica de cada data warehouse.

Definindo uma source

- Sources são definidas em arquivos YAML dentro do diretório `models/`.
- Pode ser usado:

```
models/model_properties.yml
```

- Ou qualquer outro arquivo `.yml` dentro de `models/`.
- A definição fica na seção:

```
sources:
```

- Exemplo:

```
sources:
  - name: raw
    tables:
      - name: phone_orders
      - name: web_orders
```

- Nesse exemplo:
  - `raw` é o nome da source.
  - `phone_orders` e `web_orders` são tabelas de origem.
- As opções disponíveis podem variar conforme o data warehouse.

Acessando sources

- A função `source()` recebe dois argumentos:

```
{{ source('source_name', 'table_name') }}
```

- Exemplo:

```
SELECT *
FROM {{ source('raw', 'phone_orders') }}
```

- Depois da compilação, o dbt substitui `source()` pelo nome correto da tabela conforme a configuração do warehouse.
- O resultado pode assumir uma forma como:

```
database_name.table_name
```

- O formato exato depende do data warehouse.

Exemplo com múltiplas sources

- Um modelo pode combinar várias tabelas de origem.

```
SELECT *
FROM {{ source('raw', 'phone_orders') }}
UNION ALL
SELECT *
FROM {{ source('raw', 'web_orders') }}
```

- Esse modelo poderia representar um conjunto consolidado chamado `all_orders`.

Testando sources

- Sources podem receber os mesmos tipos de testes usados em modelos:
  - Testes nativos.
  - Testes singulares.
  - Testes genéricos/reutilizáveis.
- Os testes são definidos dentro da seção `sources:` do arquivo YAML.
- Exemplo:

```
sources:
  - name: raw
    tables:
      - name: phone_orders
        columns:
          - name: order_id
            tests:
              - not_null
              - unique
```

`ref()` vs `source()`

Função	Uso
`ref()`	Referenciar outro modelo dbt
`source()`	Referenciar uma tabela de origem já existente no warehouse

Seeds no dbt

O que são dbt seeds?

- **Seeds** são arquivos **CSV** que podem ser carregados diretamente no data warehouse pelo dbt.
- Normalmente representam conjuntos de dados que mudam pouco.
- Exemplos:
  - Lista de países.
  - Códigos postais.
- Seeds **não devem** ser usados para:
  - Dados brutos.
  - Dados exportados de outro processo.

Por que usar seeds?

- Existem três motivos principais:

    - Arquivos CSV são fáceis de editar e copiar.
    - Podem ser usados em desenvolvimento, testes e produção.
    - Por serem arquivos de texto, podem ser versionados facilmente com Git.
- Isso permite:
    - Comparar versões.
    - Rastrear alterações.
    - Manter os dados junto ao código do projeto.

Como definir um seed?

- Adicione um arquivo `.csv` ao diretório:

```
seeds/
```

- O cabeçalho deve estar na primeira linha do arquivo.
- Exemplo:

```
zipcode,place,state
00501,Holtsville,NY
01001,Agawam,MA
```

- Para carregar o seed no data warehouse:

```
dbt seed
```

- Esse comando importa os arquivos CSV do diretório `seeds/`.

Configurações adicionais

- Seeds podem receber configurações como:
    - Banco de dados.
    - Schema.
    - Uso de aspas.
    - Tipos de dados das colunas.
- As configurações podem ser aplicadas:
    - Ao projeto inteiro.
    - A seeds específicos.
- Para configurações globais, pode ser usado:

```
dbt_project.yml
```

- Para configurações específicas de seeds, pode ser usado um arquivo como:

```
seeds/properties.yml
```

- O arquivo pode ter outro nome, desde que esteja no diretório `seeds/`.

Definindo tipos de dados

- Os tipos disponíveis dependem do data warehouse.
- Exemplos comuns:
  - `integer`
  - `varchar`
- Se o tipo não for definido, o dbt tenta inferi-lo a partir do conteúdo.
- Exemplo: definir `zipcode` como texto para preservar zeros à esquerda.

```
seeds:
  - name: zipcodes
    config:
      column_types:
        zipcode: varchar(5)
```

- Isso evita transformar valores como:

```
00501
```

em:

```
501
```

Testando seeds

- Seeds suportam testes assim como models e sources.
- Exemplo:

```
seeds:
  - name: zipcodes
    columns:
      - name: zipcode
        tests:
          - unique
```

- Nesse caso, o dbt verifica se os códigos postais são únicos.

Acessando seeds nos models

- Depois de carregado no warehouse, um seed pode ser referenciado com `ref()`.
- Exemplo:

```
SELECT *
FROM {{ ref('zipcodes') }}
```

- Depois de criado, o seed se comporta de forma semelhante a um model para fins de referência.

Models vs Sources vs Seeds

Objeto	Origem	Como acessar
Model	Transformação criada pelo dbt	`ref()`
Source	Dados já existentes no warehouse	`source()`
Seed	Arquivo CSV carregado pelo dbt	`ref()`

Snapshots no dbt

O que é

- Um **snapshot** registra as mudanças de um conjunto de dados ao longo do tempo.
- Ele permite visualizar:
  - Os diferentes estados de um objeto.
  - Quando cada estado foi válido.
- Exemplos:
  - Status de um pedido.
  - Etapa de produção de um carro.
  - Status de envio de uma compra.

SCD2

- Uma forma comum de acompanhar mudanças históricas em um data warehouse é a **Slowly Changing Dimension Type 2 (SCD2)**.
- SCD2 é uma técnica usada para preservar o histórico de alterações dos dados.
- É comum em data warehouses no estilo Kimball.
- No dbt, **snapshots são usados para implementar SCD2**.

Exemplo de SCD2

- Considere um pedido com três estados:
  - Recebido.
  - Empacotado.

    - Enviado.
- Se apenas atualizarmos a mesma linha, os estados anteriores são perdidos.
- Isso impede responder perguntas como:
    - Quanto tempo o pedido ficou como Recebido?
    - Quanto tempo levou para passar de Empacotado para Enviado?
- Com SCD2, cada mudança pode ser registrada como uma nova versão histórica da linha.

SCD2 no dbt

- O dbt adiciona automaticamente colunas para controlar a validade de cada versão:
Coluna	Função
`dbt_valid_from`	Data e hora em que aquela versão passou a ser válida
`dbt_valid_to`	Data e hora em que aquela versão deixou de ser válida
- Quando `dbt_valid_to` é `NULL`, aquela versão representa o estado mais recente.

Implementando snapshots

- Snapshots são definidos em arquivos `.sql` dentro do diretório:

```
snapshots/
```

- Exemplo:

```
snapshots/snapshot_orders.sql
```

- Um snapshot começa com:

```
{% snapshot snapshot_orders %}
```

- E termina com:

```
{% endsnapshot %}
```

- A configuração é definida com `config()`.

Exemplo de snapshot

```
{% snapshot snapshot_orders %}
{{
    config(
        target_schema='snapshots',
```

```
        strategy='timestamp',
        unique_key='order_id',
        updated_at='updated_at'
    )
}}
SELECT *
FROM {{ source('raw', 'orders') }}
{% endsnapshot %}
```

Configurações principais

Opção	Função
`target_schema`	Schema onde o snapshot será armazenado
`strategy`	Estratégia usada para detectar mudanças
`unique_key`	Coluna que identifica unicamente cada registro
`updated_at`	Coluna que informa quando o registro foi atualizado

`target_schema`

- É recomendado usar um schema separado do schema padrão.
- Snapshots adicionam colunas extras, o que pode afetar ferramentas existentes.

`strategy`

- A estratégia mais comum apresentada é:

```
timestamp
```

- Ela usa uma coluna de data/hora para identificar alterações.
- Outra estratégia mencionada é:

```
check_cols
```

- Ela compara determinadas colunas para identificar mudanças.

`unique_key`

- Representa a coluna que identifica cada registro.
- Normalmente corresponde a uma chave primária.

`updated_at`

- Indica a coluna que contém a data e hora da última alteração do registro.

Fonte dos dados

- Snapshots normalmente ficam próximos do início do fluxo de dados.
- Por isso, é comum acessar uma source:

```
SELECT *
FROM {{ source('raw', 'orders') }}
```

- Também é possível usar `ref()` para acessar um modelo.

Executando snapshots

- Para criar ou atualizar snapshots:

```
dbt snapshot
```

- Depois, o snapshot pode ser acessado em modelos com:

```
{{ ref('snapshot_orders') }}
```

Frequência de execução

- `dbt snapshot` precisa ser executado regularmente.
- Se o intervalo entre execuções for muito grande, algumas mudanças podem não ser registradas.
- Por isso, é recomendável automatizar ou agendar a execução do snapshot.

Revisão dos principais componentes do dbt

Sources, seeds, models e tests

- Ao trabalhar com dados no dbt, é comum começar por **sources** e **seeds**.
- **Sources** representam os dados que chegam inicialmente pelos processos de extração e carga.
- **Seeds** representam dados auxiliares, como:
  - Listas de países.
  - Códigos de área.
  - CEPs.
- **Models** são o principal componente do dbt.
- Eles realizam as transformações dos dados, normalmente a partir de sources ou seeds, para consumo a jusante.
- **Tests** podem validar vários componentes do projeto, incluindo:
  - Sources.

    - Seeds.
    - Models.
- Os tipos de tests incluem:
    - Nativos/integrados.
    - Singulares.
    - Genéricos/reutilizáveis.

`dbt build`

- O comando `dbt build` funciona como um atalho para executar várias tarefas comuns do projeto dbt.

```
dbt build
```

- Ele pode:
    - Executar models.
    - Rodar tests.
    - Processar seeds.
    - Executar snapshots, se definidos.
- Essas tarefas são organizadas como um único job.
- Isso permite tratar falhas de forma integrada durante o processo.
- `dbt build` **não executa** operações de documentação.
- Ele não substitui comandos como:

```
dbt docs generate
dbt docs serve
```

- Os comandos individuais continuam podendo ser executados separadamente.

Por que usar `dbt build`?

- Comandos individuais funcionam bem, mas podem não considerar todas as dependências do projeto como um todo.
- Por exemplo:
    - `dbt run` executa models, mas não realiza todas as validações antes das atualizações.
    - Seeds podem não estar preparados para models dependentes.
- `dbt build` considera as dependências do projeto e coordena a execução dos componentes.
- Ele é especialmente útil em ambientes de produção.
- Em ambientes de desenvolvimento ou teste, pode ser mais prático usar comandos individuais.

Uso por ambiente

Ambiente	Abordagem mais comum
Desenvolvimento	Comandos individuais
Teste	Comandos individuais
Produção	`dbt build`

- `dbt build` pode ser excessivo para pequenas alterações ou validações locais.

Opções do `dbt build`

`--select`

- Permite selecionar models ou objetos específicos para o build.

```
dbt build --select model_name
```

- É útil em projetos com muitos models.

`-d`

- Exibe informações adicionais de depuração.

```
dbt build -d
```

- Ajuda a entender melhor o que está acontecendo durante a execução.

`--exclude`

- Permite excluir objetos específicos do processo.

```
dbt build --exclude model_name
```

- Pode ser útil para evitar a execução de models grandes ou desnecessários em determinada situação.

Visão geral dos comandos

Comando	Função principal
`dbt seed`	Carrega seeds
`dbt run`	Executa models
`dbt test`	Executa tests

Comando	Função principal
`dbt snapshot`	Atualiza snapshots
`dbt build`	Coordena a execução de vários componentes
`dbt docs generate`	Gera a documentação
`dbt docs serve`	Disponibiliza a documentação localmente